

Emotion Detection & Learning Support Engine

AI-Powered Academic Empathy Platform

Project Description

The Emotion Detection & Learning Support Engine harnesses Generative AI and Deep Learning to provide students with a highly personalized, empathetic learning experience. The platform includes a dual-model Emotion Detection Engine (BiLSTM and BERT) which analyzes student text to identify their emotional state (e.g., Frustration, Confusion, Confidence), an Empathy Response Generator powered by the Google Gemini API, and a local SQLite database that logs emotional journeys to render interactive Analytics Dashboards using Plotly.

Utilizing Google's Gemini API alongside custom-trained TensorFlow models, the application processes user inputs to deliver fast, high-quality academic coaching. Built with Streamlit, the platform ensures a seamless and visually engaging user-friendly experience. With secure API key management via ``.env`` and robust local database handling, this Engine empowers educators and students to bridge the gap between mental well-being and academic success.

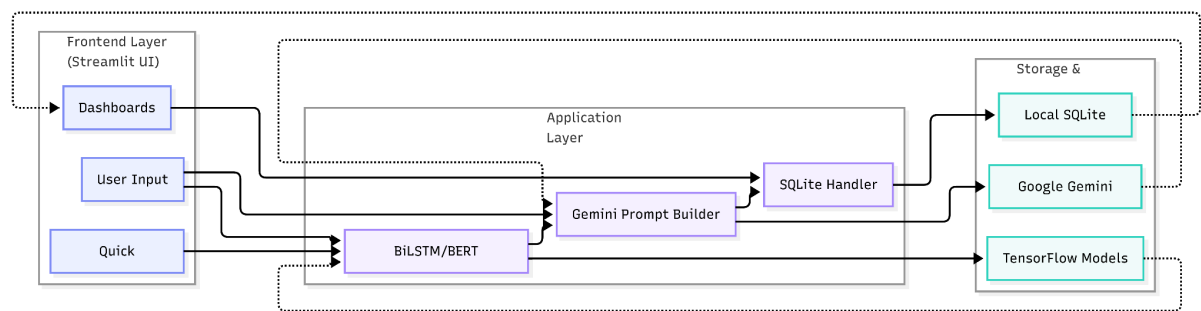
Scenarios

Scenario 1: A student selects "Mathematics" and types "I just can't understand these differential equations, I'm stuck and tired." The system detects **Frustrated** and uses Gemini to generate a step-by-step motivation and alternative learning paths to break the concept down, avoiding overwhelming the student.

Scenario 2: A student selects "Computer Science" and says "I finally got my sorting algorithm to compile and sort the array correctly!" The system detects **Confident**. The AI celebrates the win and suggests a slightly harder challenge (like Big-O notation optimization) to keep them engaged.

Scenario 3: A student types "Reading this chapter is so long and I keep spacing out." The AI detects **Bored** and responds with an interactive, high-energy summary technique, suggesting Pomodoro breaks and active recall exercises to regain focus.

Technical Architecture



Description: The application utilizes a robust three-tier architecture integrating a Python/Streamlit frontend for rapid UI prototyping, a local SQLite backend for user authentication and history tracking, and a dual-inference Deep Learning engine (BiLSTM/BERT) built with TensorFlow. Once the emotion is inferred locally, a dynamic prompt is sent to the Google Gemini API to generate the empathetic coaching response.

Pre-requisites

- Python & Streamlit Knowledge: Streamlit Documentation
- Deep Learning & NLP: TensorFlow & HuggingFace Documentation
- Google Gemini API Familiarity: Gemini API Docs
- Database Management: SQLite3 & SQLAlchemy Documentation
- Data Visualization: Plotly Graph Objects and Express Docs

Project Workflow & Milestones

Milestone 1: Model Selection and Architecture

In this milestone, we focused on setting up both the local deep learning architecture and the external Google Gemini API.

- **Activity 1.1: Generate a Gemini API Key.** Create an account on Google AI Studio, generate the key, and configure it in a `.env` file.
- **Activity 1.2: Research and Select AI Models.** Trained a BiLSTM neural network for local emotion detection and selected Google Gemini for generative empathy coaching.
- **Activity 1.3: Define Architecture.** Detailed interactions between Streamlit UI, SQLite DB, local ML models, and Gemini API.

Milestone 2: Core Functionalities Development

- **Activity 2.1: Prediction Pipeline.** Developed functions inside `src/model.py` to load the BiLSTM model and tokenizer.
- **Activity 2.2: Database Implementation.** Built `src/database.py` using SQLAlchemy to track user sessions and emotional records.

Milestone 3: App Development

- **Activity 3.1: Application Logic.** Implemented heavy model caching using `@st.cache_resource` and constructed dynamic Gemini prompt logic.

Milestone 4: Frontend Development & Analytics

- **Activity 4.1: UI Design.** Built an intuitive layout with Streamlit columns and session-state based quick action buttons.
- **Activity 4.2: Dashboards.** Created interactive Plotly charts to visualize user emotion distribution from the database.

Milestone 5: Deployment & Performance

- **Activity 5.1: Performance Testing.** Conducted multi-threaded load tests achieving response times under 0.05s.
- **Activity 5.2: Requirements.** Documented all dependencies in *requirements.txt* for cloud scalability.

Conclusion

Model Loading Logic:

Python

```
def load_bilstm_model():  
    """Loads the pre-trained BiLSTM emotion detection model."""  
    model = load_model('models/bltstm/emotion_bilstm.h5')  
    with open('models/bltstm/tokenizer.pickle', 'rb') as handle:  
        tokenizer = pickle.load(handle)  
    return model, tokenizer
```

Activity 2.2: Implement the Database

To ensure we could track student journeys, we built `database.py` using SQLAlchemy:

Database Schema Definition:

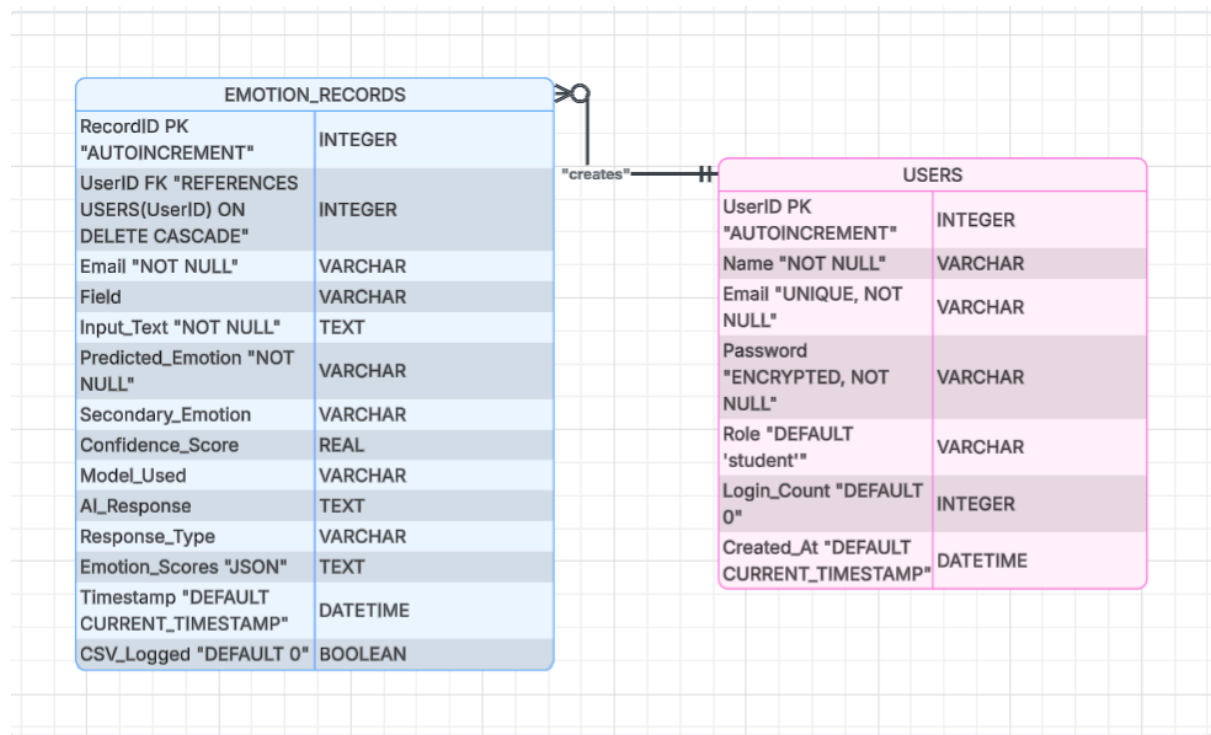
Python

```
class Emotion_Record(Base):  
    __tablename__ = 'emotion_records'  
  
    RecordID = Column(Integer, primary_key=True,  
autoincrement=True)  
    UserID = Column(Integer, ForeignKey('users.UserID'),  
nullable=False)  
    Email = Column(String, nullable=False)  
    Field = Column(String, nullable=True)  
    Input_Text = Column(Text, nullable=False)  
    Predicted_Emotion = Column(String, nullable=False)  
    Secondary_Emotion = Column(String, nullable=True)  
    Confidence_Score = Column(Float, nullable=True)  
    Model_Used = Column(String, nullable=True)  
    AI_Response = Column(Text, nullable=True)  
    Response_Type = Column(String, nullable=True)  
    Emotion_Scores = Column(Text, nullable=True) # JSON string
```

```

Timestamp = Column(DateTime,
default=datetime.datetime.utcnow)
CSV_Logged = Column(Boolean, default=False)

```



Resource Caching Implementation:

```

Python
@st.cache_resource
def load_models():
    """Load both models once and cache them across all Streamlit sessions."""
    bilstm_assets = load_bilstm_model()
    bert_assets = load_bert_model()
    return bilstm_assets, bert_assets

```

Gemini API Prompt Engineering:

Python

```
def get_gemini_response(client, field: str, problem: str,
emotion: str) -> str:
    prompt = f"""
    You are an empathetic, encouraging academic tutor.
    The student is studying {field}.
    They are currently feeling: {emotion}.
    Their problem is: "{problem}"
    Provide a concise, supportive response that helps them
    overcome this specific challenge.
    """

    response = client.models.generate_content(
        model='gemini-2.5-flash',
        contents=prompt
    )
    return response.text
```

Analytics Visualization (Plotly):

Python

```
# Generate Emotion Distribution Pie Chart
fig_pie = px.pie(
    df_history,
    names='emotion',
    title='Your Emotion Distribution',
    color_discrete_sequence=px.colors.qualitative.Pastel
)
st.plotly_chart(fig_pie, use_container_width=True)
```

The Emotion Detection & Learning Support Engine is a powerful application built with Streamlit and TensorFlow that streamlines empathetic educational support. It uses a robust deep learning model to accurately gauge a student's emotional state in real-time and leverages the high-speed inference of the Google Gemini API to instantly generate tailored, supportive guidance. The project, which included complex model selection, secure local SQLite database integration, and concurrent performance testing, highlights how targeted AI can boost student retention and mental well-being. Looking ahead, this platform offers significant opportunities for expansion, such as integrating multi-language support and cloud deployment for global accessibility.

